

Lógica Proposicional: Conceptos Fundamentales y Aplicaciones

Una guía completa sobre los fundamentos de la lógica proposicional, sus conectivas lógicas, y las técnicas de formalización y evaluación mediante tablas de verdad.



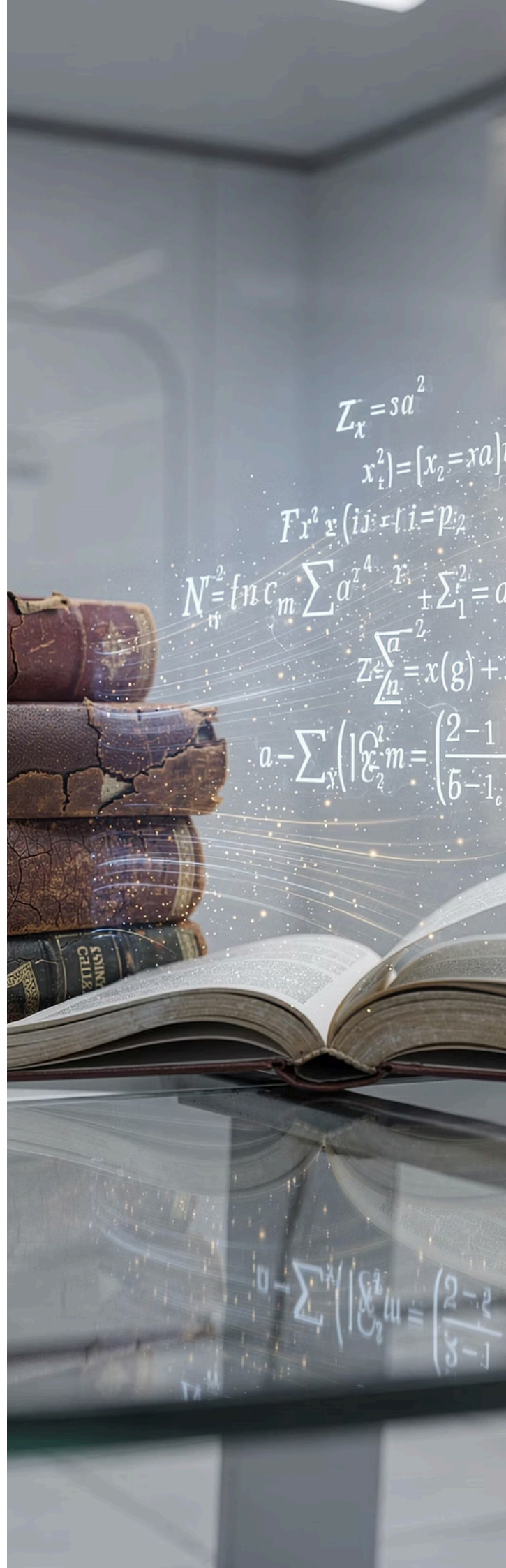
¿Qué es la Lógica?

Es la ciencia que se encarga de estudiar las formas de las estructuras o esquemas del razonamiento formal. Está establecida los principios fundamentales y proporciona los métodos necesarios que permite determinar que cierto razonamiento sea válido o no lo sea.

Lógica Proposicional

Es un sistema formal cuyos elementos más simples representan proposiciones y cuyas constantes lógicas llamadas conectivas lógicas, representan operaciones sobre proposiciones, capaces de formar otras proposiciones, capaces de formar otras proposiciones de mayor complejidad.

También se puede decir, que es la parte de la lógica que se encarga específicamente de las proposiciones.



Proposiciones: La Base del Razonamiento Lógico

Son oraciones o enunciados susceptibles de ser clasificadas, inequívocamente, de verdaderas o falsas, es decir, las proposiciones son aquellas expresiones que afirman algo, para la cual se dispone de algún criterio que nos permita establecer, sin lugar a duda, si tal afirmación es verdadera o falsa, por lo que su valor de verdad o valor veritativo ("V" o "F").

Para la lógica proposicional, una proposición puede ser el resultado o composición de otras proposiciones, la primera se llamaría proposición resultante (también proposición compuesta) y la otra proposición componente (o proposición simple).

Proposición Simple

Se denomina enunciado o proposición simple o atómica a aquel enunciado o proposición que no tiene conectores lógicos.

Proposición Compuesta

Es un enunciado o proposición que está formada por dos o más enunciados o proposiciones simples; por consiguiente, están separadas por diferentes conectores lógicos.

Variables de proposiciones

Una proposición podrá ser representada por medio de una letra minúscula (p, q, r, s y t), o también una letra minúscula con subíndice de ser necesario. Las letras o símbolos utilizados para este fin se conocen como variables proposicionales. Ejemplo:

$$\neg(p \wedge q) \rightarrow \neg r$$

Constantes Lógicas: Conectores y Operadores

En la lógica proposicional, las conectivas lógicas son tratados como funciones de verdad, es decir, como funciones que toman conjuntos de valores de verdad y devuelven valores de verdad.

El significado de las conectivas lógicas no es nada más que su comportamiento como funciones de verdad. Cada conectiva lógica se distingue de las otras por los valores de verdad que devuelve frente a las distintas combinaciones de valores de verdad que puede recibir. Esto quiere decir que el significado de cada conectiva lógica puede ilustrarse mediante una tabla que despliegue los valores de verdad que la función devuelve frente a todas las combinaciones posibles de valores de verdad que puede recibir.

¿Qué son las Conectivas?

Una *conectiva lógica* o simplemente *conectiva*, (también llamado *operador lógico* o *conectores lógicos*) es un símbolo o palabra que se utiliza para conectar dos fórmulas bien formadas o sentencias, de modo que el valor de verdad de la fórmula compuesta depende del valor de verdad de las fórmulas componentes.

Principales Conectivas Lógicas

Negación	No	$\neg \sim$	No está lloviendo
Conjunción	Y	$\wedge \&$	Está lloviendo y está nublado
Disyunción	O	$\vee$	Está lloviendo o está nublado
Condicional material	Si ... Entonces	$\rightarrow \supset$	Si está soleado, entonces es de día
Bicondicional	Si y solo si	$\leftrightarrow \equiv$	Está nublado si y solo si hay nubes visibles
Disyunción opuesta	Ni ... ni	$\downarrow$	Ni está soleado ni está nublado
Disyunción exclusiva	O bien ... o bien	$\Delta \oplus$	O bien está soleado o bien está nublado

Conectivas Fundamentales: Definiciones Detalladas

Negación ($\neg$)

Es una operación sobre la proposición donde su valor de verdad es un valor semántico. La negación de una proposición es **verdadera** cuando dicha proposición es falsa, y viceversa. Su formalización es de la forma $\neg p$.

Conjunción ($\wedge$)

Es un conector lógico binario entre dos proposiciones, cuyo valor de la verdad resulta **cierto** sólo si ambas proposiciones son verdaderas, y es **falso** de cualquier otra forma. Su formalización es de la forma $p \wedge q$.

Disyunción ($\vee$)

Es un conector lógico binario entre dos proposiciones, cuyo valor de la verdad resulta **falso** sólo si ambas proposiciones son falsas, y **cierto** de cualquier otra forma. Su formalización es de la forma $p \vee q$.

Conectivas Avanzadas

Condición material ($\rightarrow$)

También conocido como condicional funcional de verdad o como implicación material, y es una conectiva lógica que conecta dos proposiciones. El condicional es una función de verdad binaria, que se vuelve **falso** cuando B es falso siendo A verdadera, y se vuelve **verdadero** en cualquier otro caso. Su formalización es de la forma $p \rightarrow q$.

Bicondicional ($\leftrightarrow$)

También llamado equivalencia o doble implicación. Este es un operador lógico binario que conecta dos proposiciones y el valor de verdad es **verdadero** cuando ambas proposiciones tienen el mismo valor de verdad, es decir, ambas son verdaderas o falsas simultáneamente, de lo contrario es **falso**. Su formalización es de la forma $p \leftrightarrow q$.

Disyunción opuesta ($\downarrow$)

También conocidas como la negación conjunta o flecha de peirse, es una conectiva lógica cuyo valor de verdad resulta **verdadero** sólo si ambas proposiciones son falsas, y **falso** de cualquier otra forma. Su formalización es de la forma $p \downarrow q$.

Disyunción exclusiva (Δ)

También llamada exclusivo o desigualdad material, es un operador lógico entre dos proposiciones; y el valor de verdad es **verdadera** cuando ambas proposiciones tienen valores de verdad diferentes.

Formalización de Proposiciones

La operación consistente en sustituir las expresiones del lenguaje natural por símbolos lógicos se llama *formalización*. A la proposición debidamente formalizada se conocerá como *fórmula*.

¿Qué es la Formalización?

Por formalización se entienden dos aspectos relacionados:

- **Como proceso:** se refiere al proceso de traducción o simbolización de las proposiciones del lenguaje natural del lenguaje cotidiano al lenguaje lógico.
- **Como estructura:** la formalización nos permite explicitar la estructura ordenada o forma lógica de las proposiciones del lenguaje natural que se simbolizan o se traducen al lenguaje lógico.

Sintaxis

Todos los lenguajes se componen de unos símbolos y de unas reglas sintácticas que nos permite indicar que combinaciones de símbolos son correctos y cuales no los son.

Reglas para Fórmulas Bien Formuladas (FBF)

Una fórmula enunciativa es una expresan, en la que interviene variables de enunciados y conectivas que puede formarse utilizando las siguientes reglas:

01	02	03
Toda proposición es FBF	Si A es un FBF, entonces $\neg A$ también es FBF	Si A y B son FBF, entonces $(A \wedge B)$, $(A \vee B)$ y $(A \rightarrow B)$ también es FBF

Normas para la escritura de formas enunciativas

- a) Una conectiva afecta a las letras proposicionales inmediatas o a los conjuntos inmediatos a ella que están entre paréntesis.
- b) Reglas de precedencia:

Nivel 1	Nivel 2	Nivel 3	Nivel 4
$(,), [,], \{ \}$	$\neg$	$\wedge, \vee$	$\rightarrow, \leftrightarrow$

Pasos para la formalización

- Definir las proposiciones simples (p, q, r, ...)
- Formalizar según su jerarquía
- Menor jerarquía: Coma, Punto y coma
- Mayor jerarquía: Punto, Dos signos de puntuación

Ejemplos de Formalización: Conjunción y Disyunción

Formalización de la conjunción

Proposición en lenguaje natural: Los perros son listos y los gatos egoístas

p: Los perros son listos

q: Los gatos son egoístas

Formalización: $p \wedge q$

Proposición en lenguaje natural: Estudiaré, pero también veré la tele

p: Estudiaré

q: Veré la tele

Formalización: $p \wedge q$

Proposición en lenguaje natural: Además de comer tarta, beberé sidra

p: Comeré tarta

q: Beberé sidra

Formalización: $p \wedge q$

Formalización de la disyunción

Proposición en lenguaje natural: Voy al cine o al teatro

p: Voy al cine

q: Voy al teatro

Formalización: $p \vee q$

Proposición en lenguaje natural: Puedo ir al cine o no

p: Iré al cine

q: No iré al cine

Formalización: $p \vee q$

Ejemplos de Formalización: Condicional y Negación

Formalización del condicional

Proposición en lenguaje natural: Si Misha es un gato, entonces escupirá bola de pelos

p: Misha es un gato

q: Misha escupirá bola de pelos

Formalización:

$p \rightarrow q$

Proposición en lenguaje natural: Si vas a la playa, te broncearás

p: Vas a la playa

q: Te broncearás

Formalización:

$p \rightarrow q$

Proposición en lenguaje natural: Pégame y tendrás tu merecido

p: Pégame

q: Tendrás tu merecido

Formalización:

$p \rightarrow q$

Proposición en lenguaje natural: Asistir a clase es condición

necesaria para aprobar

p: Se asiste a clase

q: Se aprueba

Formalización:

$q \rightarrow p$

Formalización de la negación

Proposición en lenguaje natural: No voy a solucionarte el problema

p: Voy a solucionarte el problema

Formalización: $\neg p$

Proposición en lenguaje natural: No es cierto que haya estado en ese cine

p: He estado en ese cine

Formalización: $\neg p$

Formalizaciones Combinadas: Ejemplos Complejos

También se puede realizar *formalizaciones combinadas*, como se muestra a continuación:

Proposición en lenguaje natural: No voy a ir a París, pero si voy, me acordaré de ti y de tu madre.

p: Voy a París

q: Me acordaré de ti

r: Me acordaré de tu madre

Formalización: $\neg p \wedge [p \rightarrow (q \wedge r)]$

Proposición en lenguaje natural: Si vas al cine, entonces, o compras palomitas o me enviarás si tienes hambre.

p: Vas al cine

q: Compras palomitas

r: Me envías

s: Tienes hambre

Formalización: $p \rightarrow [q \vee (s \rightarrow r)]$

Proposición en lenguaje natural: Me quieres o no, tendrás que soportarme

p: Me quieres

q: Tienes que soportarme

Formalización: $(p \vee \neg p) \rightarrow q$

Proposición en lenguaje natural: Si en Marte no hay agua, entonces no hay vida; en consecuencia, no hay marcianos ni platillos voladores

p: Marte hay agua

q: Marte hay vida

r: Hay marcianos

s: Hay platillos voladores

Formalización: $(\neg p \rightarrow \neg q) \rightarrow (\neg r \wedge \neg s)$ ó $(\neg p \rightarrow \neg q) \rightarrow (\neg r \downarrow s)$

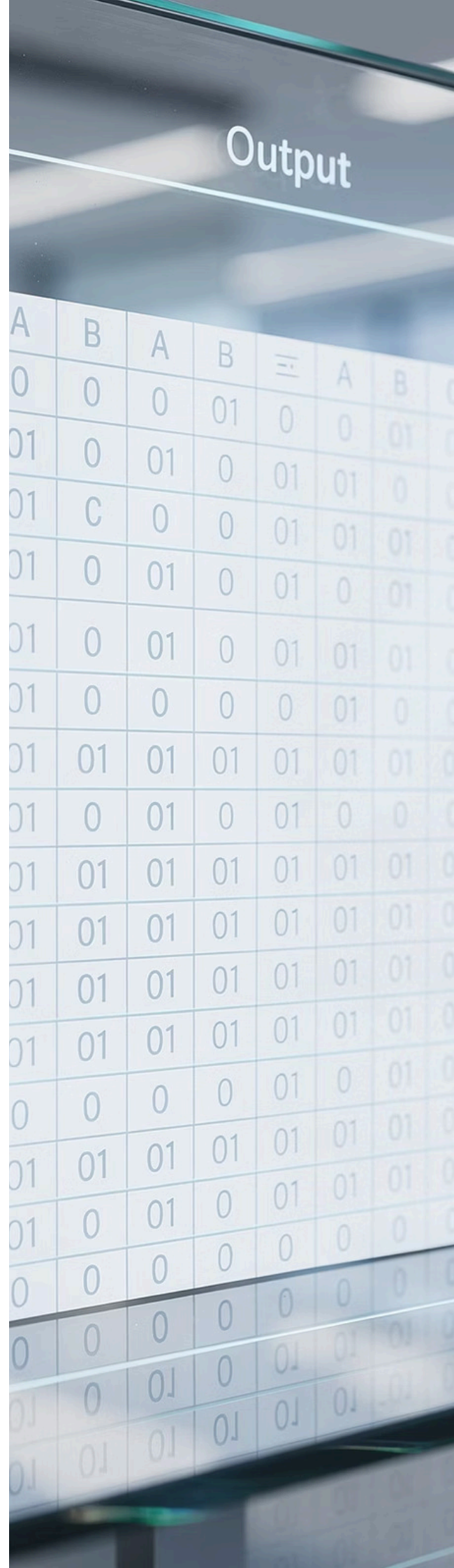
Tabla de la Verdad: Fundamentos

Una tabla de verdad o tabla de valores de verdad, es una tabla que muestra el valor de verdad de una proposición compuesta, para cada combinación de verdad que se pueda asignar a sus componentes. También se puede decir, que es una estrategia de la lógica simple que permite establecer la validez de una o varias proposiciones compuestas en cuanto a cualquier situación, osea, determinan las condiciones necesarias para que sea verdadero la proposición propuesta. Cada proposición posee 2 valores de verdad, los cuales son **verdadero** o **falso**, en el cual se podrá definir en la tabla de verdad como "V" y "F". Estudiaremos la tabla de verdad de cada uno de los conectivos ante mencionado.

Cálculo del número de filas

Para elaborar una tabla de verdad se consideró que existe n proposiciones, en el cual para determinar el número de filas a utilizar se recurrió a una pequeña fórmula que es:

$$N^{\circ} \text{ de Fila} = 2^n, \text{ donde } n = \\ N^{\circ} \text{ de proposiciones}$$



Tablas de Verdad de las Conectivas Principales

Tabla de verdad de Negación

Como ya se había mencionado anteriormente, la negación es un operador unitario debido a que solo afecta el valor de verdad de una proposición. Está es la tabla de verdad más sencilla de realizar debido a que solo se puede obtener 2 valores de verdad, los cuales serán su valor negado u opuesto de su valor de verdad.

V	F
F	V

Ejemplo: No es cierto que Rodrigo es maestro

p: Rodrigo es maestro

Formalización: $\neg p$

Tabla de verdad de Conjunción

En la conjunción existe claramente un valor de verdad *verdadero*, y este se debe cuando ambas proposiciones son ciertas, de lo contrario es *falso*, es decir, que para el valor de verdad sea cierto no puede existir ninguna falsa.

V	V	V
V	F	F
F	V	F
F	F	F

Ejemplo: Juan es futbolista y Ana es voleibolista

p: Juan es futbolista

q: Ana es voleibolista

Formalización: $p \wedge q$

Tabla de verdad de Disyunción

Este conector también es binario, por lo que se deberá realizar similar al anterior, pero en este caso se debe tener en cuenta que el único valor de verdad *falso* es cuando ambas proposiciones son falsas, de lo contrario es *verdadera*, es decir, que para el valor de verdad *verdadero* al menos uno de las proposiciones debe ser verdadero.

V	V	V
V	F	V
F	V	V
F	F	F

Ejemplo: Raúl es ingeniero o es profesor

p: Raúl es ingeniero

q: Raúl es profesor

Formalización: $p \vee q$

Tablas de Verdad Completas y Clasificación

Tabla resumen de todas las conectivas

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$	$p \oplus q$	$p \downarrow q$
V	V	F	V	V	V	F	V	F
V	F	F	F	V	F	F	F	V
F	V	V	F	V	V	F	F	V
F	F	V	F	F	V	V	V	F

Clasificación de la tabla de verdad

Las tablas de verdad se pueden clasificar según su resultado de las siguientes formas:

Tautología
Es aquella proposición compuesta que es cierta para todos los valores de verdad que se asignen a cada una de las proposiciones, independientemente de los valores de verdad de las proposiciones que la componen, es decir, que todos los valores de verdad sean "V".

Contradicción
Una proposición compuesta corresponde a una contradicción cuando los valores de verdad que se asignen a cada una de las proposiciones son falsas sin importar el valor de las proposiciones que la forman, es decir, que todos los valores de verdad sean "F".

Contingencia
Una proposición compuesta cuyos valores en sus diferentes líneas de la tabla dan como resultado "V" y "F" se conoce como contingencia, inconsistencia o falacia. Tecnológicamente, las contingencias se utilizan para construir circuitos de control y automatismo.

Ejemplo de Tautología

Hallar la tabla de la verdad del siguiente esquema proposicional:

$$[(p \vee q) \wedge \neg q] \rightarrow p$$

p	q	$\neg q$	$p \vee q$	$(p \vee q) \wedge \neg q$	$[(p \vee q) \wedge \neg q] \rightarrow p$
V	V	F	V	F	V
V	F	V	V	V	V
F	V	F	V	F	V
F	F	V	F	F	V

Se puede concluir, que la proposición es **Tautológica** ya que todos sus valores de verdad son **verdaderos**.

Ejemplo de Contradicción

Hallar la tabla de la verdad del siguiente esquema proposicional:

$$(p \wedge q) \wedge \neg p$$

p	q	$\neg p$	$p \wedge q$	$(p \wedge q) \wedge \neg p$
V	V	F	V	F
V	F	F	F	F
F	V	V	F	F
F	F	V	F	F

Se puede concluir, que la proposición es **Contradictorial** ya que todos sus valores de verdad son **falsos**.

Ejemplo de Contingencia

Hallar la tabla de la verdad del siguiente esquema proposicional:

$$(p \rightarrow q) \wedge (q \rightarrow r)$$

p	q	r	$p \rightarrow q$	$q \rightarrow r$	$(p \rightarrow q) \wedge (q \rightarrow r)$
V	V	V	V	V	V
V	V	F	V	F	F
V	F	V	F	V	F
V	F	F	F	V	F
F	V	V	V	V	V
F	V	F	V	F	F
F	F	V	V	V	V
F	F	F	V	V	V

Se puede concluir, que la proposición es **Contingente** ya que sus valores de verdad contienen tanto **verdaderos** como **falsos**. Este tipo de proposiciones son útiles en circuitos de control donde se necesitan diferentes salidas según las condiciones de entrada.

Propiedades de la Lógica Proposicional

Las propiedades lógicas son reglas algebraicas que nos permiten **simplificar y transformar** expresiones lógicas complejas en formas equivalentes más simples. Son fundamentales para optimizar circuitos digitales, simplificar código y demostrar equivalencias lógicas.

¿Por qué son importantes?

- Permiten reducir expresiones complejas a formas más manejables
- Facilitan la demostración de equivalencias lógicas
- Son la base para la optimización de circuitos digitales
- Ayudan a encontrar formas normales (CNF y DNF)

A continuación, exploraremos las principales propiedades organizadas por categorías.



Propiedades Básicas: Identidad, Dominación y Idempotencia



Propiedades de Identidad

Elemento neutro que no altera el resultado:

- $p \wedge V \equiv p$ (La conjunción con verdadero preserva p)
- $p \vee F \equiv p$ (La disyunción con falso preserva p)

Utilidad: Eliminar elementos redundantes en expresiones lógicas.



Propiedades de Dominación

Elemento absorbente que determina el resultado:

- $p \wedge F \equiv F$ (Cualquier cosa AND falso es falso)
- $p \vee V \equiv V$ (Cualquier cosa OR verdadero es verdadero)

Utilidad: Simplificar expresiones cuando aparecen constantes lógicas.



Propiedades de Idempotencia

Repetir la misma proposición no cambia el resultado:

- $p \wedge p \equiv p$
- $p \vee p \equiv p$

Utilidad: Eliminar duplicados en expresiones lógicas.

Propiedades Estructurales: Conmutativa, Asociativa y Distributiva

Propiedad Conmutativa

El orden de los operandos no altera el resultado:

- $p \wedge q \equiv q \wedge p$
- $p \vee q \equiv q \vee p$

Utilidad: Reordenar términos para facilitar la simplificación o agrupación.

Propiedad Asociativa

El agrupamiento de operaciones no afecta el resultado:

- $(p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$
- $(p \vee q) \vee r \equiv p \vee (q \vee r)$

Utilidad: Cambiar la forma de agrupar términos sin alterar el significado.

Propiedad Distributiva

Una operación se puede distribuir sobre otra:

- $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$
- $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$

Utilidad: Expandir o factorizar expresiones para encontrar formas equivalentes más simples. Fundamental para convertir a formas normales.

Leyes de De Morgan y Propiedades de Negación

Leyes de De Morgan

Fundamentales para transformar negaciones de expresiones compuestas:

- $\neg(p \wedge q) \equiv \neg p \vee \neg q$
 - "No (A y B)" = "No A o No B"
- $\neg(p \vee q) \equiv \neg p \wedge \neg q$
 - "No (A o B)" = "No A y No B"

Utilidad: Mover negaciones hacia adentro de expresiones complejas, esencial para simplificación y forma normal negativa (NNF).

Doble Negación

- $\neg(\neg p) \equiv p$

Utilidad: Eliminar negaciones redundantes.

Complemento

- $p \wedge \neg p \equiv F$ (Contradicción)
- $p \vee \neg p \equiv V$ (Tercero excluido)

Utilidad: Identificar contradicciones y tautologías en expresiones.

Propiedades de Absorción e Implicación

Leyes de Absorción

Permiten eliminar términos redundantes:

- $p \vee (p \wedge q) \equiv p$
 - Si p es verdadero, toda la expresión es verdadera
- $p \wedge (p \vee q) \equiv p$
 - Si p es falso, toda la expresión es falsa

Utilidad: Simplificar expresiones eliminando subexpresiones redundantes. Muy útil en optimización de circuitos.

Propiedades de la Implicación

Transformaciones del condicional:

- $p \rightarrow q \equiv \neg p \vee q$ (Definición del condicional)
- $p \rightarrow q \equiv \neg q \rightarrow \neg p$ (Contraposición)
- $\neg(p \rightarrow q) \equiv p \wedge \neg q$ (Negación del condicional)

Utilidad: Convertir implicaciones a formas con AND/OR/NOT para simplificación. La contraposición es útil en demostraciones matemáticas.

Propiedad del Bicondicional

- $p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p)$
- $p \leftrightarrow q \equiv (p \wedge q) \vee (\neg p \wedge \neg q)$

Utilidad: Descomponer bicondicionales en formas más simples.

Ejemplo Práctico de Simplificación

Veamos cómo aplicar estas propiedades para simplificar una expresión compleja paso a paso.

Expresión original:

$$(p \wedge q) \vee (p \wedge \neg q) \vee (\neg p \wedge q)$$

Proceso de simplificación:

Paso 1: Aplicar distributiva para factorizar p

$$= p \wedge (q \vee \neg q) \vee (\neg p \wedge q)$$

Paso 2: Aplicar complemento ($q \vee \neg q \equiv V$)

$$= p \wedge V \vee (\neg p \wedge q)$$

Paso 3: Aplicar identidad ($p \wedge V \equiv p$)

$$= p \vee (\neg p \wedge q)$$

Paso 4: Aplicar distributiva

$$= (p \vee \neg p) \wedge (p \vee q)$$

Paso 5: Aplicar complemento ($p \vee \neg p \equiv V$)

$$= V \wedge (p \vee q)$$

Paso 6: Aplicar identidad ($V \wedge x \equiv x$)

$$= p \vee q$$

Resultado: La expresión compleja se simplifica a $p \vee q$

Aplicación práctica: En diseño de circuitos digitales, esta simplificación reduce 3 compuertas lógicas a solo 1, ahorrando componentes y energía.

Introducción a la Lógica de Predicados

Hasta ahora, nuestras proposiciones eran "bloques cerrados" (p, q) . Pero, ¿qué pasa si queremos decir "Todos los usuarios tienen contraseña"? La lógica proposicional se queda corta porque no puede "mirar dentro" de los grupos.

Aquí entra la **Lógica de Predicados** (o de primer orden), que nos permite expresar propiedades sobre conjuntos de objetos y hacer generalizaciones.

Componentes principales:

- **Variables y Constantes:** x, y (variables que representan objetos), o "Juan" (constante)
- **Predicados:** Funciones que describen una propiedad. Ejemplo: $\text{Admin}(x)$ significa "x es administrador"
- **Dominio:** El conjunto de todos los objetos posibles (ej. "Todos los usuarios registrados en la base de datos")
- **Cuantificadores:** Símbolos que nos permiten generalizar sobre el dominio



Cuantificadores: Universal y Existencial

Los cuantificadores son los símbolos que nos permiten generalizar sobre conjuntos de objetos.

Cuantificador Universal ($\forall$): "Para todo..."

Indica que la propiedad se cumple para todos los elementos del dominio.

- **Notación:** $\forall x, P(x)$
- **Ejemplo:** "Para todo usuario x, x tiene un ID único"
- **En programación:** Equivale a recorrer una lista con un bucle for y verificar que la condición se cumpla en cada iteración. Si falla en uno, la proposición es falsa.

Cuantificador Existencial ($\exists$): "Existe al menos un..."

Indica que la propiedad se cumple para alguno (uno o más) de los elementos.

- **Notación:** $\exists x, P(x)$
- **Ejemplo:** "Existe un usuario x tal que x tiene permisos de super-administrador"
- **En programación:** Equivale a buscar en una lista hasta encontrar el primero que cumpla la condición (find o any).

Negación de Cuantificadores

La negación de cuantificadores sigue reglas específicas que transforman un cuantificador en su opuesto:

Negación del Universal

$$\neg(\forall x, P(x)) \equiv \exists x, \neg P(x)$$

- "No es cierto que todos cumplan P" equivale a "Existe al menos uno que NO cumple P"
- **Ejemplo:** "No todos los usuarios tienen contraseña" = "Existe al menos un usuario sin contraseña"

Negación del Existencial

$$\neg(\exists x, P(x)) \equiv \forall x, \neg P(x)$$

- "No existe ninguno que cumpla P" equivale a "Todos NO cumplen P"
- **Ejemplo:** "No existe ningún usuario menor de edad" = "Todos los usuarios son mayores de edad"

Traducción de Especificaciones a Lógica de Predicados

El analista de sistemas debe traducir frases de negocio a lógica de predicados para diseñar las reglas de la base de datos.

Ejemplo práctico:

Cliente dice:

"Ningún usuario menor de edad puede comprar alcohol"

Traducción a lógica:

$$\forall x (Usuario(x) \wedge Menor(x) \rightarrow \neg PuedeComprarAlcohol(x))$$

Lectura:

"Para todo x, si es usuario Y es menor, entonces NO puede comprar alcohol"

Proceso de traducción:

1. Identificar el cuantificador (¿habla de "todos", "algún", "ningún"?)
2. Identificar los predicados (propiedades: Usuario, Menor, PuedeComprar)
3. Identificar las conectivas lógicas (Y, O, SI...ENTONCES, NO)
4. Construir la fórmula respetando la precedencia de operadores

Modelización en Pseudocódigo

La teoría lógica es el plano; la programación es la construcción. El pseudocódigo es un lenguaje intermedio entre el humano y la máquina.

Ejemplo de regla de decisión:

```
SI (usuario_activo == VERDADERO) Y ((saldo > 0) O  
(credito_aprobado == VERDADERO)) ENTONCES  
    Autorizar_Compra()  
SINO  
    Rechazar_Compra()  
FIN SI
```

Observaciones importantes:

Paréntesis: Sin ellos, la lógica sería ambigua. Los paréntesis definen el orden de evaluación.

Precedencia de operadores: El AND ($\wedge$) suele "pegar" más fuerte que el OR ($\vee$), es una regla matemática que debemos respetar.

Estructura clara: El pseudocódigo permite visualizar la lógica antes de implementarla en un lenguaje específico.

□ Fórmula lógica equivalente:

$$(usuarioactivo \wedge (saldo positivo \vee creditoaprobado)) \rightarrow autorizar$$

Implementación en R: Lógica Vectorizada

R es un lenguaje muy potente para el análisis de datos, y maneja la lógica de una forma particular: **¡Vectorizada!** Esto significa que puede aplicar operaciones lógicas a listas enteras de datos de una sola vez, similar a la lógica de predicados.

Operadores lógicos en R:

- AND se escribe &
- OR se escribe |
- NOT se escribe !
- Los valores de verdad son TRUE y FALSE

Ejemplo 1: Evaluación simple

```
# Definimos un vector de usuarios (su edad)
edades <- c(15, 20, 17, 35, 12)

# ¿Quién es mayor de edad (>= 18)?
es_mayor <- edades >= 18

# Resultado: [1] FALSE TRUE FALSE TRUE FALSE
```

Ejemplo 2: Conjunción (AND)

```
membresia <- c(TRUE, TRUE, FALSE, TRUE, FALSE)

# Aplicamos la Conjunción (AND)
beneficio <- es_mayor & membresia

# R compara posición a posición:
# Usuario 1: F & V -> FALSE
# Usuario 2: V & V -> TRUE <-- ¡Este recibe beneficio!
# Usuario 3: F & F -> FALSE
```

Aplicación práctica:

Esta capacidad de R para operar sobre conjuntos de datos utilizando lógica booleana es lo que lo hace ideal para filtrar Big Data. Cuando haces un filtro en un DataFrame, estás aplicando pura lógica proposicional.

